

Conversation AI Agent Harness

Objective

Design and implement an **Agent Harness** that powers a HighLevel Conversation AI agent end-to-end. The harness receives an inbound customer message, decides what the agent should do , answers from a knowledge base, executes a skill, or hands it over to a human , executes it, and replies back into the CRM conversation with full transparency into every step of the execution.

The Challenge

Production conversation agents fail in predictable ways: they retrieve knowledge on every turn and get slow, they scatter provider-specific logic across the codebase, they update the wrong CRM fields, and nobody can explain *why* the agent said what it said. Your goal is to build a harness that gets these fundamentals right , **multi-provider, skill-driven, selectively retrieving, measurable, and fully traceable** , and backed with evals rather than demos alone.

Requirements

1. Setup & Integration

- Use a sandbox account from the HighLevel Marketplace.
- Interact with the CRM through the **inbound message webhook** (receive customer messages) and the **Conversations Send Message API** (send agent replies into the same conversation). Contact updates go through the Contacts API.
- Handle webhook realities: duplicate deliveries (idempotency) and rapid back-to-back customer messages.

2. Core Functionality Your harness must support four capabilities:

- **Multi-Provider LLM Support:**
 - Run on **Claude, OpenAI, and Gemini** behind a single abstraction , switching providers is a config change, not a code change.
 - Normalize tool/function-calling, streaming, and error semantics across providers.
- **Knowledge Base Integration (RAG-triggered):**
 - Ingest a small knowledge base (10–20 docs for a fictional business); chunk, embed, and index it.
 - The agent must decide *when* to trigger retrieval , chit-chat and skill-only turns should not hit the vector store. Answers must be grounded in retrieved content; when the KB has no answer, say so or hand over , never make up information.
- **Skills (as an extensible framework , adding a skill is registration, not core changes):**
 - **Update Contact Field:** extract details the customer shares (name, email, budget, preferred time) and update the contact via the CRM API.

- **Human Handover:** detect explicit requests, frustration, or out-of-scope topics; stop the bot for that conversation, mark it in the CRM via adding a tag or change owner assignment, and send an appropriate final message.
- **Appointment Booking:** book appointments using the HighLevel Calendars public APIs - fetch free slots for a calendar, offer them to the customer, handle their pick (including relative asks like “tomorrow afternoon”), and create the appointment against the contact. Handle no-availability and slot-taken races gracefully.
- **Execution Transparency:**
 - Every turn produces an inspectable trace: assembled prompt, provider/model used, whether RAG triggered (chunks + scores), skills fired (inputs, outputs, CRM calls), tokens, and per-step latency. A reviewer should answer “why did the agent say that?” in under a minute.

3. Quality Bars

- **Latency:** target $p50 \leq 3s$ / $p95 \leq 6s$ webhook-to-send for non-RAG turns. Measure and report actuals per provider.
- **Evals (required):** a one-command eval suite covering RAG trigger precision/recall, answer groundedness, both skills (including negative cases that must NOT fire), and latency benchmarks across all three providers. Aim for 20–30 cases per behavior and report failures.

Deliverables

1. Code & Implementation

- Github repo URL for the harness (preferred language/stack is typescript).
- Documented steps to install, connect to a HighLevel sandbox, and run the agent end-to-end.
- The eval suite and the command to reproduce the results.

2. Demo Video

- **Workflow:** show a real conversation flowing through the CRM:
 - **Knowledge:** a KB question answered with grounding , and one the agent correctly declines.
 - **Skills:** a contact field update and a human handover, visible in the CRM.
 - **Trace:** walk through one execution trace and one provider switch.
- You can use tools like Loom to record the demo.

3. Documentation

- A brief README describing your architecture, key design decisions and trade-offs, and how you handled “**Team of One**” ownership (Product, Design, Engineering & QA).
- Eval results table (per provider) with a candid failure analysis.
- Notes on what is functional vs. what is mocked.

Evaluation Criteria

- **Architecture & Abstractions:** is adding a fourth provider or a third skill obviously cheap? Are harness, providers, skills, CRM client, and tracing cleanly separated?
- **Completeness:** does the system close the loop from inbound webhook to grounded reply, skill execution, and handover in a real sandbox?
- **Eval Rigor:** reproducible results, negative cases included, honest failure analysis , evidence over vibes.
- **Technical Integrity:** clarity of the orchestration loop, RAG-triggering logic, and latency engineering.
- **Manual Code Review:** we urge only non-slop code to be submitted after a thorough manual review. AI tools are encouraged.